Phishing URL Detection Using Python

README — Project Documentation in PDF Format

Date: 21 October 2026

1. Introduction

Phishing is one of the most common cyberattacks used to steal usernames, passwords, banking details, OTPs, and personal information. Attackers create fake websites that look similar to trusted websites such as banks, social-media platforms, online stores, or email services. Users may click a malicious link received through email, SMS, WhatsApp, or social media and unknowingly enter confidential details on a fake page.

This project provides a Python-based Phishing URL Detection System. Instead of relying only on a blacklist of known malicious websites, the system analyzes the structure and behavior of a URL. It extracts important features and uses a machine-learning model to decide whether the link is likely to be phishing or legitimate.

The main advantage of this approach is that it can identify suspicious patterns even when the exact phishing URL has not been previously reported. This makes the system more useful than a simple blacklist-based checker.

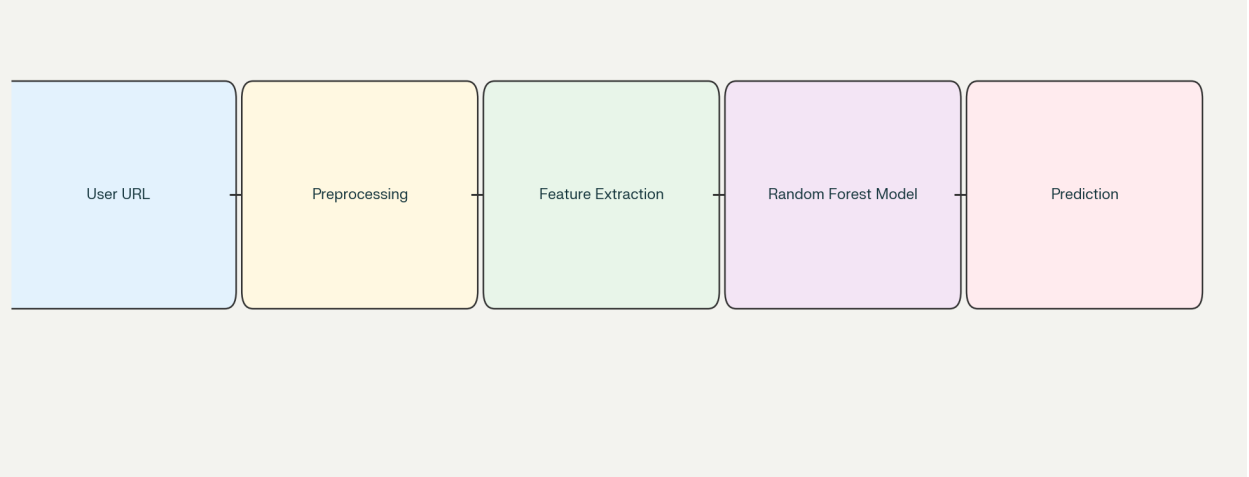
2. Project Objective

The objective of this project is to build an automated system that can analyze a URL and classify it as phishing or legitimate. The project focuses on accuracy, simplicity, and practical usability.

- Detect phishing URLs automatically using Python.
- Extract important URL-based security features.
- Train a Random Forest machine-learning classification model.
- Evaluate the model using accuracy, precision, recall, F1-score, and confusion matrix.
- Save the trained model for future predictions.
- Provide a reusable prediction function for new URLs.
- Support future integration with Flask, Streamlit, or a browser extension.

3. System Architecture

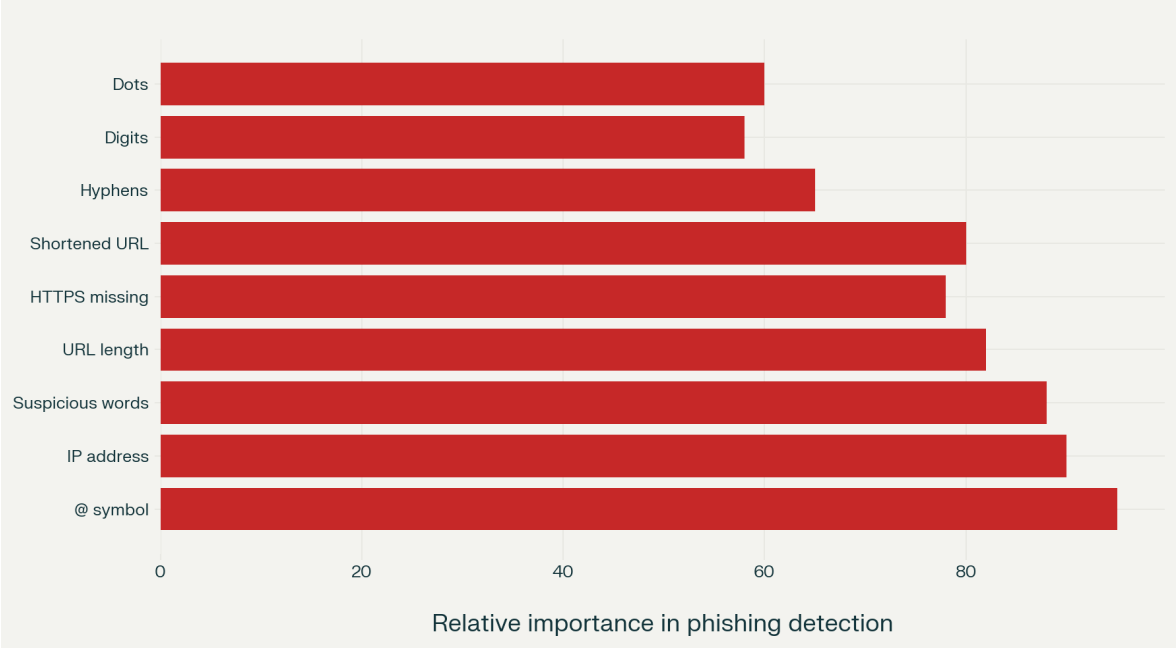
The system follows a simple machine-learning pipeline. First, the user provides a URL. The program cleans and processes the URL, extracts useful features, and sends those features to the trained Random Forest model. Finally, the model returns a prediction: Phishing URL or Legitimate URL.



The architecture is modular, so each part can be improved separately. For example, new features such as domain age, SSL certificate details, DNS records, or webpage content can be added later without changing the complete project.

4. Important URL Features

Phishing URLs often contain unusual patterns. The model uses several features to identify these patterns. No single feature is enough to confirm phishing, but together they help the machine-learning model make a stronger prediction.

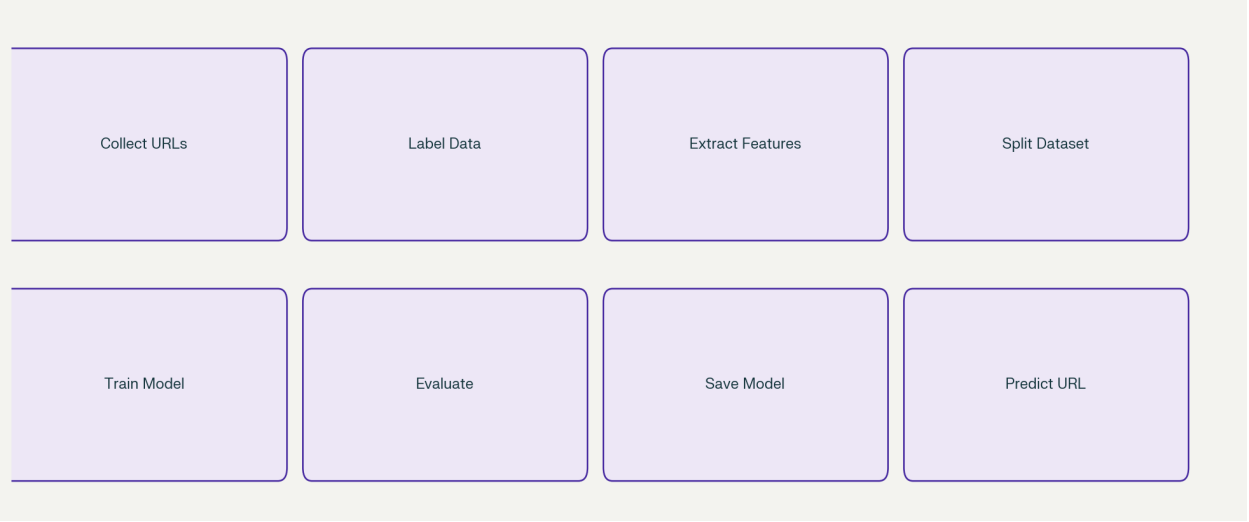


- **URL length:** Phishing URLs are often longer because attackers add extra words to look trustworthy.
- **HTTPS missing:** Many phishing sites use HTTP instead of HTTPS, although HTTPS alone does not guarantee safety.
- **IP address:** A URL using an IP address instead of a domain name can be suspicious.
- **@ symbol:** An @ symbol can mislead users about the actual destination website.
- **Excessive hyphens:** Fake domains often use many hyphens, such as secure-bank-login-verify.com.
- **Too many dots:** Multiple subdomains can hide the real domain.

- **Suspicious keywords:** Words such as login, verify, secure, account, update, and confirm are common in phishing links.
- **Shortened URL:** Short links can hide the actual destination from the user.
- **High digit count:** Automatically generated malicious URLs may contain many numbers.

5. Project Workflow

The project follows a standard supervised machine-learning workflow. Data is collected and labeled, useful features are extracted, the dataset is divided into training and testing parts, and the model is trained and evaluated.



After training, the model is saved using joblib. This allows the program to load the trained model later without training it again. The prediction function can then check any new URL entered by the user.

6. Technologies Used

This project uses Python because it has strong support for data science, machine learning, web development, and automation.

Technology	Purpose
Python 3	Main programming language
pandas	Reading, cleaning, and processing the dataset
NumPy	Numerical operations and array handling
scikit-learn	Machine-learning model, training, and evaluation
joblib	Saving and loading the trained model
requests	Optional web requests and URL checking
tldextract	Extracting domain, subdomain, and top-level domain
python-whois	Optional domain registration and age checking
dnspython	Optional DNS-related analysis
BeautifulSoup4	Optional webpage HTML content analysis
matplotlib	Charts, graphs, and diagrams
Flask	Optional lightweight web application
Streamlit	Optional interactive dashboard

7. Folder Structure

```
phishing-url-detection/
■
```

```
■■■ phishing_url_detector.py      # Main Python program
■■■ phishing_dataset.csv          # URL dataset with features and labels
■■■ phishing_url_model.pkl        # Saved trained machine-learning model
■■■ requirements.txt              # Required Python packages
■■■ README.pdf                    # Project documentation
■■■ images/                        # Diagrams and screenshots
```

8. Installation Guide

Follow these steps to set up the project on your computer.

```
# 1. Create a project folder
mkdir phishing-url-detection
cd phishing-url-detection

# 2. Create a virtual environment
python -m venv venv

# 3. Activate the virtual environment
# Windows
venv\Scripts\activate

# Linux / macOS
source venv/bin/activate

# 4. Install required packages
pip install -r requirements.txt

# 5. Run the project
python phishing_url_detector.py
```

9. Dataset Format

The dataset should be stored in CSV format. Each row represents one URL, and each column represents one extracted feature. The final column, label, is the target value.

```
url_length,num_dots,num_hyphens,has_at_symbol,uses_https,has_ip,num_digits,has_suspicious_word,num
_slashes,hostname_length,has_shortener,label
25,2,0,0,1,0,0,0,2,14,0,0
38,3,2,1,0,0,4,1,3,22,0,1
```

In the dataset, use **1** for phishing URLs and **0** for legitimate URLs. A larger and balanced dataset gives better results. Ideally, include several thousand examples of both phishing and legitimate URLs.

10. Sample Python Code

The following code shows the main part of the project: feature extraction, model training, evaluation, and prediction.

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

data = pd.read_csv("phishing_dataset.csv")
X = data.drop(columns=["label"])
y = data["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

joblib.dump(model, "phishing_url_model.pkl")

model = joblib.load("phishing_url_model.pkl")
print(model.predict([[38, 3, 2, 1, 0, 0, 4, 1, 3, 22, 0]]))
```

11. Model Evaluation

Model evaluation is important because a model with high accuracy may still miss many phishing URLs. Therefore, the project uses multiple evaluation metrics.

Metric	Meaning
Accuracy	Percentage of total URLs classified correctly
Precision	Of all URLs predicted as phishing, how many were actually phishing
Recall	Of all actual phishing URLs, how many were detected
F1-score	Balance between precision and recall
Confusion Matrix	Shows correct and incorrect predictions in detail

12. Advantages

- Automatically checks URLs before the user visits a website.
- Can detect suspicious patterns in new or unknown URLs.
- Faster than manual URL inspection.
- Can be integrated into browsers, email systems, and mobile apps.
- Reduces the risk of password, banking, and personal-data theft.
- Easy to improve by adding new security features.

13. Limitations

- Attackers constantly create new phishing patterns.
- A safe-looking URL may still redirect to a malicious website.
- The model depends on the quality and size of the dataset.
- HTTPS does not always mean a website is safe.
- Some legitimate websites may be marked as phishing due to unusual URL structures.

14. Future Enhancements

- Add domain age checking using WHOIS data.
- Check SSL certificate validity and issuer details.
- Analyze webpage content for fake login forms.
- Use deep-learning models for URL sequence analysis.
- Create a browser extension for real-time warnings.
- Add a Flask or Streamlit web interface.
- Continuously update the dataset with newly reported phishing URLs.

15. Conclusion

This project shows how Python and machine learning can be used to detect phishing URLs. By analyzing URL structure and security-related features, the system can warn users before they visit a potentially dangerous website. With a larger dataset, additional domain and webpage features, and continuous model updates, this project can become a practical cybersecurity tool for students, developers, and organizations.